



WAYPOINT CONNECT

SYSTEM ARCHITECTURE DOCUMENTATION

System Architecture

Architecture, data flows, access control and workflows

Project WaypointConnect — delivery planning and tracking for Waypoint Group's Fresh, Style and Tech brands

Event Tech-Triathlon 2026 · Team Dark Horse

Source Generated 2026-10-04 · repository commit c1e3eb6 + working tree

Accuracy rule: every statement is taken from the project files. Anything that could not be verified is marked "Not verified from the provided project files". No passwords or secrets are included.

Contents

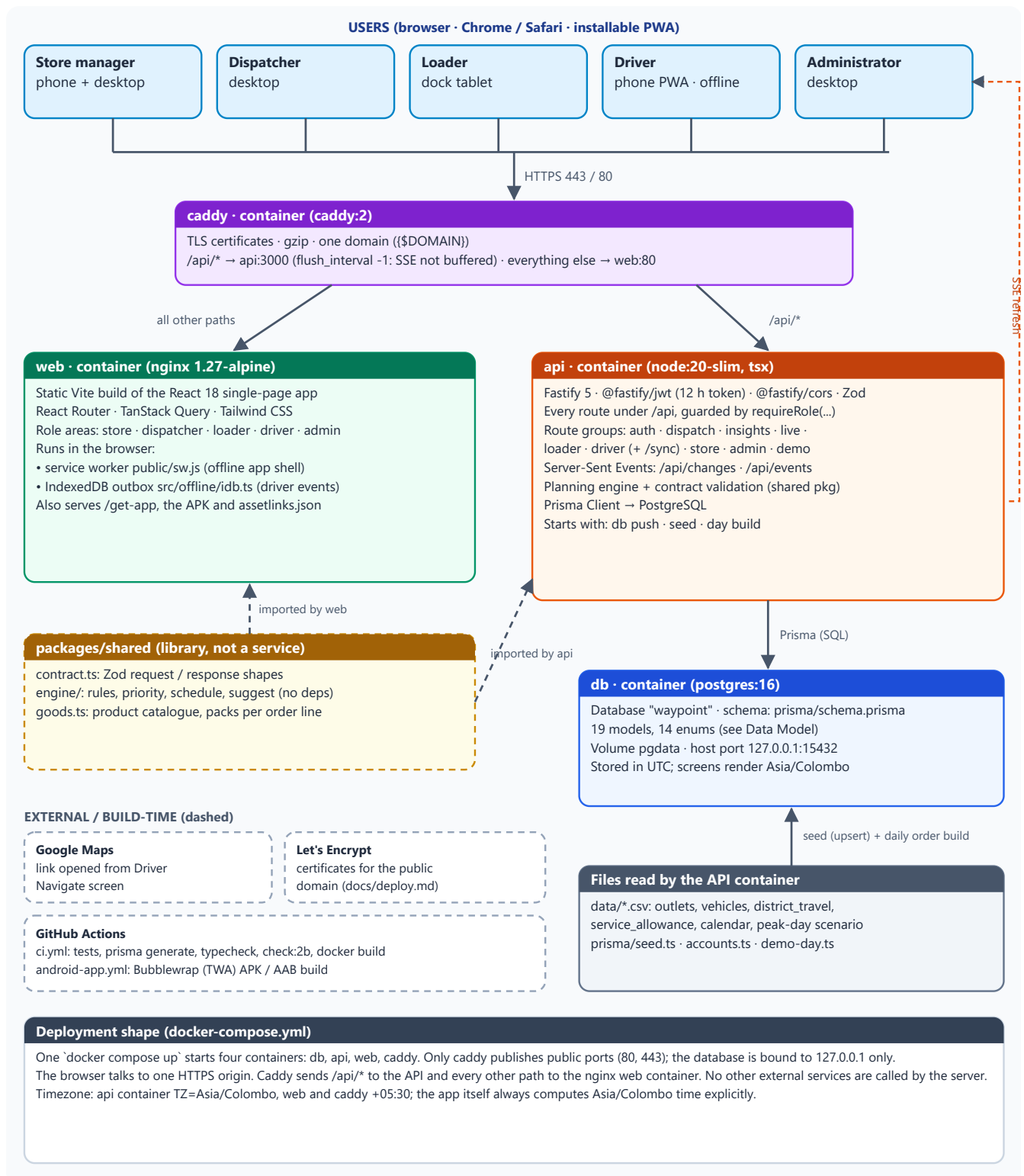
1. System overview and architecture	2
1.1 System overview	2
1.2 High-level architecture	3
2. Frontend architecture	5
2.1 Frontend architecture	5
3. Backend architecture	6
3.1 Backend architecture	6
4. Database architecture	7
4.1 Database architecture	7
5. Authentication architecture	8
5.1 Authentication architecture	8
6. API architecture	9
6.1 API architecture	9
7. Docker architecture	12
7.1 Docker architecture	12
8. Deployment architecture	13
8.1 Deployment architecture	13
9. Role-based access control	14
9.1 Role-based access control	14
10. Data flows	15
10.1 Important data flows	15
11. Order lifecycle	16
11.1 Order lifecycle	16
12. Role workflows	17
12.1 Dispatcher workflow	17
12.2 Loader workflow	17
12.3 Driver workflow	17
12.4 Store manager workflow	17
12.5 Administrator workflow	17

1. System overview and architecture

1.1 System overview

Waypoint Connect is a pnpm monorepo with three workspaces: `apps/web` (React + Vite), `apps/api` (Fastify) and `packages/shared` (the API contract and the planning engine). At runtime four containers run behind one HTTPS domain. Users reach everything through the Caddy reverse proxy: requests to `/api/*` go to the API, everything else to the nginx container that serves the built web app.

1.2 High-level architecture



Runtime architecture: users, Caddy, web and API containers, PostgreSQL and the files read at start-up. Dashed items are libraries or external/build-time components.

Component	Technology	Responsibility
Users / browsers	Chrome, Safari; installable PWA; Android TWA	Five role areas of one single-page app

Component	Technology	Responsibility
caddy	caddy:2	HTTPS certificates, gzip, routing of <code>/api/*</code> to the API (no buffering for SSE) and everything else to the web container
web	nginx 1.27-alpine serving the Vite build	Static app, cache rules (no-cache for <code>/sw.js</code> and <code>index.html</code>), <code>/get-app</code> , APK and <code>assetlinks.json</code> , <code>/version.json</code>
api	Node 20 + Fastify 5 (run with tsx)	All business logic and routes under <code>/api</code> ; imports <code>@waypoint/shared</code> ; talks to PostgreSQL through Prisma
db	PostgreSQL 16	All persistent state; schema from <code>prisma/schema.prisma</code>
packages/shared	TypeScript, Zod	<code>contract.ts</code> (every request/response shape) and <code>engine/</code> (all planning rules; no npm dependencies so it runs in the browser, the API and the tests)
data/ and prisma/ files	CSV and TypeScript	Reference data and seed. The API reads the CSVs at start-up for travel times, handling allowances and the calendar; the seed upserts outlets, vehicles, drivers and accounts

2. Frontend architecture

2.1 Frontend architecture

- **Single-page app** (`apps/web/src/App.tsx`) with React Router. Routes: `/login` , `/get-app` , and one guarded area per role: `/dispatcher/*` , `/loader/*` , `/driver/*` , `/store/*` , `/admin/*` . A `Guard` component redirects to `/login` without a session and to the user's own area when the role does not match.
- **Role folders:** `src/roles/<role>/` hold the screens of one role; shared UI is in `src/ui/` (Badge, Icon, Meter, Modal, States, Toast, format helpers); offline helpers are in `src/offline/` (`idb.ts` , `image.ts` , `sw-register.ts`).
- **Data access:** TanStack Query hooks per role (`roles/*/api.ts`) call `/api` through `src/api.ts` , which keeps the signed-in session (token and user) in the browser's `localStorage` . The Vite dev server proxies `/api` to `http://localhost:3000` .
- **Real-time refresh:** `src/live.tsx` opens the `/api/changes` event stream and refetches when the server signals a change; it catches up after a dropped connection or when the tab returns. A build id with `/version.json` shows a "New version available · Update" bar after a deploy.
- **Offline driver app:** the service worker caches the app shell; the driver's route lives in IndexedDB; every action is an event with a UUID that is applied to the phone's copy and queued in an outbox, then sent in order to `POST /api/sync` . Driver screens never call the API directly (CLAUDE.md rule).
- **Design tokens:** colours, fonts and spacing come from the Tailwind theme tokens copied from Figma (CLAUDE.md rule).
- **Contract:** request and response types are imported from `@waypoint/shared/contract` , so the UI cannot invent fields.

3. Backend architecture

3.1 Backend architecture

The API (`apps/api/src/server.ts`) is a single Fastify process. Plugins: `@fastify/cors` (any origin), `@fastify/jwt` (secret from `JWT_SECRET`). The body limit is 8 MB because driver sync batches carry small photos and signatures. One error shape is used everywhere: `{ "error": "plain sentence", ...details }`.

Request pipeline

1. `onRequest` hook: `ensureToday()` builds today's and the working run's orders if they do not exist yet (once per day and process).
2. Route `preHandler`: `requireRole(...)` verifies the JWT, checks the account is still active and the role matches.
3. Route handler: validates the body with Zod schemas from the contract, uses Prisma and the planning engine.
4. `onResponse` hook: any successful non-GET request (except `/api/auth/*`) calls `signalChange(role)` , which pushes a "something changed" signal to every connected screen.

Source modules

File	Responsibility
<code>auth.ts</code>	<code>requireRole()</code> route guard
<code>day.ts</code>	Loads one delivery day for one depot; maps database rows to engine types and DTOs; order display references (<code>ORD-25xxx</code>)
<code>plans.ts</code>	Writes suggested plans and trips, deferrals, publishing (plan version, snapshot, dock bays, carry-over orders, notifications)
<code>ops.ts</code>	Shared reads for the dock (loader) and the road (driver)
<code>reference.ts</code>	CSV reference data, calendar, cutoff, <code>workingDay()</code> , role clocks
<code>demo.ts</code>	Admin-controlled Demo mode (state stored in the <code>Setting</code> table)
<code>events.ts</code>	In-process event bus for the two SSE streams
<code>routes/*.ts</code>	auth, dispatch, insights, live, loader, driver, store, admin (+ demo), realtime

Planning engine (packages/shared/src/engine)

`rules.ts` (hard rules and `validatePlan`), `priority.ts` (priority policy), `schedule.ts` (stop order by closing time, ETAs, departure times), `suggest.ts` (Suggest plan: places orders in priority order, keeps reefers for chilled goods and vans for van-only outlets, defers what cannot legally fit as `unavoidable` with the reason), `reference.ts` (CSV parsing), `types.ts` . The engine has no npm dependencies.

4. Database architecture

4.1 Database architecture

PostgreSQL 16 in the `db` container, accessed only through Prisma Client 6. The schema (`prisma/schema.prisma`) has **19 models** and **14 enums**. Docker start-up runs `prisma/pre-push.sql` (adds `Vehicle.driverId` and its unique index safely on an older database), then `prisma db push --skip-generate`, then the idempotent seed. The database stays on UTC; timestamps are shown in Sri Lanka time by the app. The full description and ER diagram are in the Data Model document.

5. Authentication architecture

5.1 Authentication architecture

- **Sign-in:** `POST /api/auth/login` with email and password returns `{ token, user }`. The token is a JWT signed with `JWT_SECRET`, valid for 12 hours, whose payload is the session user: `id, name, role, depot, outletId, vehicleId`.
- **Password storage:** `User.passwordHash` holds a bcrypt hash. Seeded accounts use passwords derived from `ACCOUNT_SECRET` (HMAC of the email, 12 characters), recomputed by the seed on every start unless an administrator set a custom password (`customPassword`).
- **Authorisation:** `requireRole(...roles)` verifies the token, loads `User.active` (a switched-off account gets 401 at its next request) and answers 403 when the role is not allowed.
- **Event streams:** browsers cannot send headers to `EventSource`, so `/api/changes` and `/api/events` take the token in the query string; `/api/changes` also checks the account is active.
- **Session in the browser:** kept in `localStorage` by `src/api.ts`. The driver app can add a local fingerprint / Face ID app lock (WebAuthn); it is not server authentication.
- **Admin actions:** create staff accounts and reset passwords (a random password is generated and shown once, bcrypt-hashed), switch accounts on and off; an administrator cannot switch themselves off and the last active administrator cannot be switched off.

Observed deviation from the documented design

The login handler in `apps/api/src/routes/auth.ts` contains:

```
const isDemoPw = body.data.password === 'waypoint-demo' || ... ||
body.data.password.trim().length > 0;
const valid = isDemoPw || (await bcrypt.compare(body.data.password, user.passwordHash));
```

Because the last operand is true for any non-empty string, the bcrypt comparison is never decisive: any non-empty password is accepted for an existing account. The handler also maps `dispatcher@`, `loader@`, `driver@` and `store@` aliases to specific accounts and falls back to the first user of a role when the email prefix matches. The README and the AI disclosure describe unique per-account passwords, so the documented behaviour and the source differ. This is reported in the Testing document as a finding.

6. API architecture

6.1 API architecture

- REST-style JSON over HTTPS. **Every route lives under `/api`** (registered with prefix `/api`); Caddy forwards `/api/*` to the API container.
- Request and response shapes come only from `packages/shared/src/contract.ts` (Zod). Invalid input answers 400 with a readable message.
- Dispatcher screens work on one day and one depot: `dayOf()` takes the depot from the account; the `?depot=` query applies only to an account without a fixed depot. Loaders use their account depot; drivers use their vehicle's depot; stores are scoped to `outletId`.
- The driver's write path is a single idempotent endpoint, `POST /api/sync`.
- 56 routes in total (listed below, taken from `apps/api/src/routes` and `server.ts`).

Method	Path	Access	Purpose
POST	<code>/api/auth/login</code>	public	Sign in, returns token and session user
GET	<code>/api/health</code>	public	Database ping and server time
GET	<code>/api/auth/me</code>	any signed-in role	Current session user
GET	<code>/api/changes?token=</code>	any signed-in, active	SSE "something changed" signal for every screen
GET	<code>/api/events?token=</code>	any valid token	SSE stream of dispatcher events
GET	<code>/api/orders</code>	dispatcher	Order queue (D2)
GET	<code>/api/board</code>	dispatcher	Planning board data (D3)
POST	<code>/api/plans/suggest</code>	dispatcher	Auto-allocate (Suggest plan)
PUT	<code>/api/plans/trips</code>	dispatcher	Save the board after a drag and drop
POST	<code>/api/plans/check-placement</code>	dispatcher	Check one drop against the rules
POST	<code>/api/orders/:id/defer</code>	dispatcher	Defer an order with a reason
POST	<code>/api/deferrals/:id/confirm</code>	dispatcher	Confirm a second deferral
POST	<code>/api/plans/publish</code>	dispatcher	Publish the plan (raises the version)
GET	<code>/api/overview</code>	dispatcher	Overview (D1)
GET	<code>/api/deferrals</code>	dispatcher	Deferral review (D4)
POST	<code>/api/deferrals/options/:id</code>	dispatcher	Take a recovery option
GET	<code>/api/forecast</code>	dispatcher	Capacity forecast (D6)
GET	<code>/api/live</code>	dispatcher	Live tracking (D5)
GET	<code>/api/live/pod/:stopId</code>	dispatcher	Proof of delivery for a stop

Method	Path	Access	Purpose
POST	/api/live/alerts/:id/action	dispatcher	Act on an alert
POST	/api/live/simulate	dispatcher	Demo-only simulation (needs DEMO_MODE=1)
GET	/api/loader/vehicles	loader	Vehicles on the published plan (L1)
GET	/api/loader/trips/:id	loader	Goods lines of a trip (L2)
POST	/api/loader/trips/:id/claim	loader	Claim a trip to load
POST	/api/loader/lines/:id/tick	loader	Tick a goods line
POST	/api/loader/trips/:id/problems	loader	Report a problem (L3)
POST	/api/loader/trips/:id/seal	loader	Seal and release (L5)
GET	/api/loader/changes	loader	Plan changes v1 → v2 (L4)
POST	/api/loader/changes/:orderId/moved	loader	Mark an order moved
POST	/api/loader/changes/ack	loader	Acknowledge the new plan version
GET	/api/driver/route	driver	The vehicle's route, trips and stops
POST	/api/sync	driver	Idempotent batch of offline events
GET	/api/store/today	store	Today screen (SM1)
GET	/api/store/catalogue	store	Catalogue and last order for ordering (SM2)
POST	/api/store/catalogue/products	store	Add a product to the catalogue
POST	/api/store/orders	store	Place or replace an order
GET	/api/store/orders	store	Order history (SM7)
GET	/api/store/orders/:id	store	Order status and tracking (SM3/SM4)
POST	/api/store/orders/:id/receipt	store	Confirm receipt (SM5)
POST	/api/store/orders/:id/deferral	store	Accept or cancel a deferral (SM6)
GET	/api/store/receipts	store	Receipts (SM8)
GET	/api/store/notifications	store	Notifications
POST	/api/store/notifications/read	store	Mark notifications read
GET	/api/admin/system	admin	System health and today per depot
POST	/api/admin/system/rebuild-today	admin	Build today's orders
GET	/api/admin/accounts	admin	All accounts and counts
POST	/api/admin/accounts	admin	Create a staff account
PATCH	/api/admin/accounts/:id	admin	Switch on/off, rename
POST	/api/admin/accounts/:id/reset-password	admin	Reset a password (shown once)

Method	Path	Access	Purpose
GET	/api/admin/fleet	admin	Vehicles and drivers
POST	/api/admin/drivers	admin	Add a driver
PUT	/api/admin/vehicles/:id/driver	admin	Assign a driver to a vehicle
GET	/api/admin/demo	admin	Demo mode status and story
POST	/api/admin/demo/start	admin	Start or extend demo mode
POST	/api/admin/demo/reset	admin	Reset demo data
POST	/api/admin/demo/end	admin	End demo mode

7. Docker architecture

7.1 Docker architecture

`docker-compose.yml` defines four services and two named volumes (`pgdata` , `caddy_data`):

Service	Image / build	Ports	Notes
db	<code>postgres:16</code>	<code>127.0.0.1:\${DB_HOST_PORT:-15432}</code> →5432 (this machine only)	Database <code>waypoint</code> ; password <code>\${DB_PASSWORD:-waypoint}</code> ; health check with <code>pg_isready</code> ; stays on UTC
api	build <code>apps/api/Dockerfile</code> (<code>node:20-slim</code>)	internal 3000	Depends on a healthy db; environment: <code>DATABASE_URL</code> , <code>JWT_SECRET</code> , <code>ACCOUNT_SECRET</code> , <code>DEMO_*</code> , <code>API_PORT</code> , <code>TZ=Asia/Colombo</code> ; <code>restart: unless-stopped</code>
web	build <code>apps/web/Dockerfile</code> (build stage <code>node:20-slim</code> , runtime <code>nginx:1.27-alpine</code>)	internal 80	Serves the Vite build; files made world-readable so <code>/sw.js</code> does not answer 403
caddy	<code>caddy:2</code>	80, 443	<code>Caddyfile</code> mounted read-only; <code>DOMAIN</code> defaults to <code>localhost</code> ; certificate data in <code>caddy_data</code>

API start-up command (`apps/api/Dockerfile`):

```
pnpm prisma db execute --schema prisma/schema.prisma --file prisma/pre-push.sql \
&& pnpm prisma db push --skip-generate \
&& pnpm db:seed \
&& pnpm --filter @waypoint/api start
```

The image copies `prisma/` , `data/` , `packages/shared` , `apps/api` and `scripts/credentials.ts` , so `pnpm credentials` works inside the container. `.dockerignore` keeps `node_modules` , `dist` , `.git` , `.env` and `credentials` out of the images.

Local override file

The untracked file `docker-compose.override.yml` in the working copy additionally publishes PostgreSQL on host port 5432 so an API running outside Docker can connect. It is a local development convenience, not part of the tracked deployment.

8. Deployment architecture

8.1 Deployment architecture

- **Single host:** the whole system is one `docker compose up` on one Linux server. Only Caddy publishes public ports (80/443).
- **HTTPS:** Caddy obtains a certificate for `DOMAIN` on the first request (`docs/deploy.md` names Let's Encrypt). Without a domain, `<ip-with-dashes>.sslip.io` is used.
- **One-command install:** `scripts/server-setup.sh` (per `docs/deploy.md` and `docs/handoff.md`) installs Docker, adds swap, clones the repository to `/opt/waypoint` , writes `.env` with random secrets, runs `docker compose up -d --build` and waits for `/api/health` .
- **Live instance:** <https://3-106-77-179.sslip.io>. The AI disclosure records a deployment to AWS Lightsail with images built on a Mac for x86 because the 512 MB server cannot build them. Reachable (HTTP 200) on 2026-10-04.
- **Android app:** built by the manual GitHub Actions workflow `android-app.yml` (Bubblewrap, Trusted Web Activity) and published as `/downloads/waypoint-connect.apk` with `/.well-known/assetlinks.json` . The APK is a wrapper around the live site.
- **Team database access:** through an SSH tunnel to `127.0.0.1:15432` on the server; the port is never open to the internet.

9. Role-based access control

9.1 Role-based access control

Role	API guard	Data scope	UI guard
dispatcher	<code>requireRole('dispatcher')</code>	An account with a depot is locked to it; the account without one (<code>dispatcher@waypoint.lk</code>) chooses Peliyagoda or Kandy with the switcher	Guard <code>role="dispatcher"</code> on <code>/dispatcher/*</code>
loader	<code>requireRole('loader')</code>	The loader's depot; <code>findTrip</code> refuses a trip of another depot ("not on your depot plan")	<code>/loader/*</code>
driver	<code>requireRole('driver')</code>	The vehicle in the token (<code>vehicleId</code>) and its depot	<code>/driver/*</code>
store	<code>requireRole('store')</code>	The outlet in the token (<code>outletId</code>); every route is scoped to it	<code>/store/*</code>
admin	<code>requireRole('admin')</code>	All accounts, drivers, system and demo	<code>/admin/*</code>

A request from a signed-in account of another role answers **403** ("This action is for ... accounts"); a missing or expired token answers **401**; a switched-off account answers 401 at its next request. The web guard additionally redirects a user who opens another role's URL to their own area.

10. Data flows

10.1 Important data flows

Offline driver sync

1. The driver acts on the phone (arrive, deliver, fail). The app creates an event `{ uuid, type, time, ... }`, applies it to the cached route and stores it in the IndexedDB outbox. This works with no signal.
2. When online (on reconnect and every 20 seconds) the outbox is sent in order to `POST /api/sync`.
3. The server stores one `DeliveryEvent` per `eventId` (the primary key); events already stored are skipped, so a retry after a dropped connection never delivers twice. The response lists accepted, duplicate and rejected events and the current plan version.
4. The phone removes sent events. If the plan version changed, it refetches `GET /api/driver/route` and shows the "Back online" sheet (what was sent, what changed).

Real-time refresh

After any successful non-GET request the API emits a change signal on an in-process bus; every open `/api/changes` stream receives `{ at, by }` (no business data). Each screen refetches what it shows with the user's own access. The server also signals at the 16:00 run switch and at midnight (30-second check in `server.ts`).

Plan publishing

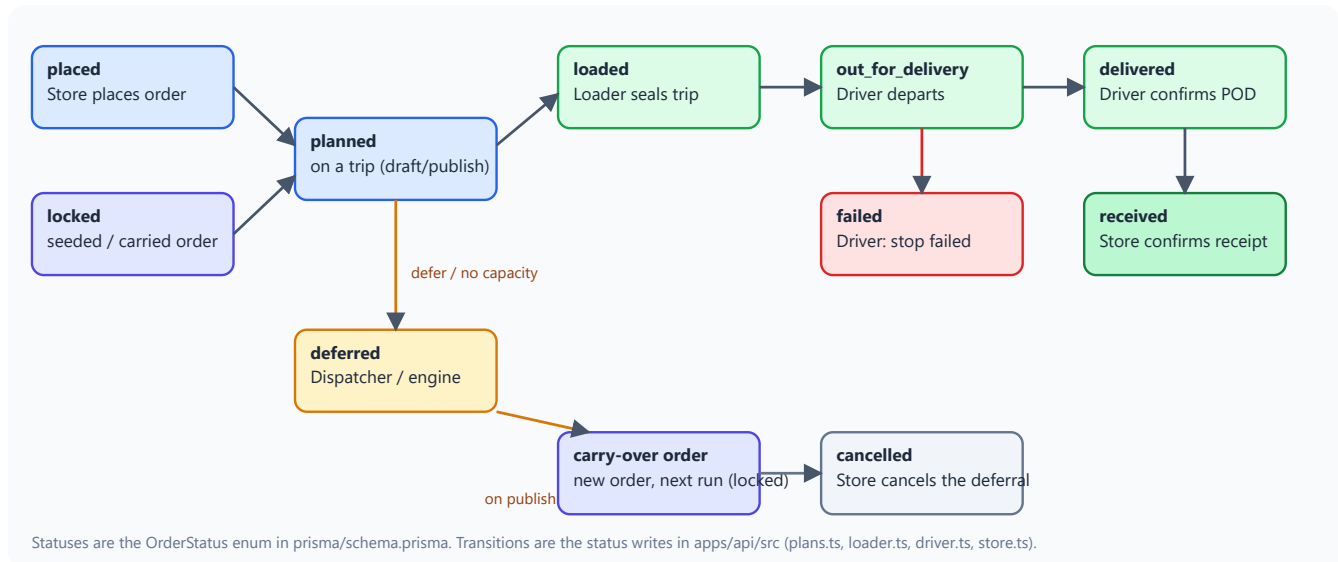
`publish()` in `plans.ts` refuses a plan that breaks a rule, leaves an order neither planned nor deferred, or skips a shop twice without confirmation. It then raises `Plan.version`, stores a `PlanVersion` snapshot, assigns dock bays (six bays, in departure order), creates carry-over orders for deferrals, notifies stores and alerts the dispatcher.

Goods in packs

Store orders, loader lists and driver deliveries are all built from the same catalogue in `packages/shared/src/goods.ts`, so a pack ordered by a store is the pack the loader ticks and the driver delivers.

11. Order lifecycle

11.1 Order lifecycle



Order statuses and who moves an order between them.

Status	Set when	By
placed	A store places or replaces an order (<code>POST /api/store/orders</code>)	Store manager
locked	Seeded scenario order; carry-over order created at publish or seal	Seed / API
planned	The order is placed on a trip	Dispatcher / engine
deferred	Suggest plan or "defer" could not or chose not to serve it	Dispatcher / engine
loaded	The trip is sealed	Loader
out_for_delivery	The driver confirms the load and departs	Driver
delivered / failed	Stop delivered or failed (via <code>/api/sync</code>)	Driver
received	Store confirms receipt	Store manager
cancelled	Store cancels a deferred order's new-day delivery	Store manager

12. Role workflows

12.1 Dispatcher workflow

1. Overview: orders, reefer space, what needs attention.
2. Order Queue: filters, greyed-out after-cutoff orders, **Auto-allocate & open board**.
3. Planning Board: drag stops; blocked drops explain the rule; meters per trip.
4. Deferrals: review reasons (`unavoidable` / `choice`), try recovery options A–C, confirm second deferrals, **Confirm & publish plan**.
5. Live Tracking: ETA vs window, alerts (Notify store, Reply to store, Move to next run), View POD.
6. Capacity Forecast for the next three weeks.

12.2 Loader workflow

1. Choose vehicle (published plan only; banner when republished).
2. Load goods: tick every line, last stop first; ! reports a problem (carry over / replace / hold).
3. Plan changes: mark moved, acknowledge the new version.
4. Seal & release: seal number (and reefer temperature for reefers). Short packs become a next-run order.

12.3 Driver workflow

1. Today's runs, then Load check: **Confirm load & depart** (only after the loader has sealed).
2. Route progress: next stop, ETA against the window, Run / Stops / Sync tabs.
3. At each stop: arrival, Start unloading, proof of delivery (photo, receiver, signature) or Report a problem.
4. Offline: actions queue on the phone; Sync tab shows waiting events; Back online shows what was sent.

12.4 Store manager workflow

1. Today: cutoff countdown, today's deliveries, tomorrow's order cards.
2. Place order (dry, and chilled for Fresh stores) from the catalogue; copy last order; edit until 16:00.
3. Track an order: Received → Scheduled → Loaded → Delivered, ETA, truck.
4. Confirm receipt line by line (OK / Short / Damaged); answer a deferral (accept or cancel).

12.5 Administrator workflow

1. System: health, working day and clock, today per depot; *Build today's orders* if a day is empty.
2. Vehicles & drivers: assign one driver per vehicle (confirm a move), add drivers.
3. Accounts: search, switch on/off, reset password (shown once), create dispatcher / loader / administrator accounts.
4. Demo mode: start for a chosen length, extend, reset demo data, end.